

ABC's of Estimating Equations



Paul Zivich,¹ Rachael Ross,²

¹University of North Carolina, ²University of Utah

Acknowledgements

Funding: K01AI177102 (PNZ), K99DA061935 (RKR),
R01AI157758 (PNZ)

Disclaimer: views are ours and not those of NIH, DHHS, US
government



pzivich@unc.edu



pzivich



pausalz@bsky.social

 github.com/pzivich/ABCs_of_M-estimation

- Open your preferred statistical software
- Open corresponding `mean.*` script
- Run the full script

Closed-form: 8.0

Root-finder: 8.0

95% CI: [0.8, 15.2]

Overview

A Terminological Note

Framework covered today goes by many names

- Estimating Equations
- M-estimation
- Z-estimation

May use terms interchangeably

Why Estimating Equations?

Learning estimating equations during my postdoc fundamentally changed how I think about and do epidemiology

- Approach problems from a different perspective

Made my work simpler by

- Making it easier to construct novel estimators
- Simplifying variance estimation¹
- Being better equipped to read more theoretical papers
- Giving me a tool set to prove statistical properties

¹I almost never use the bootstrap anymore!

Metrika

<https://doi.org/10.1007/s00184-024-00962-4>



Variance estimation for average treatment effects estimated by g-computation

Stefan Nygaard Hansen¹ · Morten Overgaard¹

Received: 3 February 2023 / Accepted: 8 March 2024

© The Author(s) 2024

Why Estimating Equations?

Assume now that an estimator $\hat{\beta}_n(\mathbf{z})$ of $\dot{\beta}(\mathbf{z})$ exists for all $\mathbf{z}$. The asymptotic covariance matrix of Theorem 2 may then be estimated by the following plug-in estimator

$$\hat{\Gamma}_n^{\mathbf{a}} = \frac{1}{n} \sum_{i=1}^n \left\{ \mu(\hat{\beta}_n; \mathbf{X}_i^{\mathbf{a}}) - \hat{\theta}_n^{\mathbf{a}} + \left(\frac{1}{n} \sum_{j=1}^n \frac{\partial}{\partial \beta} \mu(\hat{\beta}_n; \mathbf{X}_j^{\mathbf{a}}) \right) \hat{\beta}_n(\mathbf{Z}_i) \right\}^{\otimes 2} \quad (8)$$

where $\mathbf{x}^{\otimes 2} = \mathbf{x}\mathbf{x}^T$ for a column vector $\mathbf{x}$.

Under some mild regularity conditions on the estimator $\hat{\beta}_n$, this plug-in estimator will be consistent for the asymptotic covariance matrix as the following result shows.

Theorem 3 *Make the assumptions of Theorem 2 and assume furthermore that $\hat{\beta}_n$ satisfies*

$$\|\hat{\beta}_n(\mathbf{z}) - \dot{\beta}(\mathbf{z})\| \leq g_n \cdot f(\mathbf{z}) \quad (9)$$

for a sequence of random variables $g_n \xrightarrow{P} 0$ and a measurable function f with $E(f(\mathbf{Z})^2) < \infty$. Then $\hat{\Gamma}_n^{\mathbf{a}} \xrightarrow{P} \Gamma^{\mathbf{a}}$.

Proof See the Appendix. □

Why Estimating Equations?

As an alternative to the two-step approach of this paper, one could consider formulating the two steps as two estimating equations and use (stacked) M-estimation. The sandwich variance estimator from the stacked M-estimation approach corresponds to the variance estimator of this paper. This M-estimation approach has been implemented in the Python library `delicatessen` as pointed out by a reviewer.

Estimating Equations Use-Cases

Causal inference

- Reifeis et al. (2020) 'Assessing exposure effects on gene expression' *Genet Epidemiol*
- Wiener et al. (2026) 'Causal identification conditions for the effect of treatment in the treated: Illustration using the Northwest Germany Stroke Registry' *Epidemiol*
- Tchetgen Tchetgen et al. (2024) 'Universal difference-in-differences for causal inference in epidemiology' *Epidemiol*
- Zivich et al. (2023) 'Introducing proximal causal inference for epidemiologists' *Am J Epidemiol*
- Zivich et al. (2024) 'Empirical sandwich variance estimator for iterated conditional expectation g-computation' *Stat Med*

Sensitivity analysis

- Cole et al. (2023) 'Higher-order evidence' *Eur J Epidemiol*
- Cole et al. (2023) 'Sensitivity analyses for means or proportions with missing outcome data' *Epidemiol*

Measurement error

- Ross et al. (2024) 'Leveraging External Validation Data: The Challenges of Transporting Measurement Error Parameters' *Epidemiol*

Estimating Equations Use-Cases

Target trial emulation

- DeMonte et al. (2024) 'Assessing COVID-19 Vaccine Effectiveness in Observational Studies via Nested Trial Emulation' *arXiv:2403.18115*

Generalizability / transportability

- Dahabreh, et al. (2020) 'Extending inferences from a randomized trial to a new target population' *Stat Med*
- Dahabreh, et al. (2023) 'Sensitivity analysis using bias functions for studies extending inferences from a randomized trial to a target population' *Stat Med*
- Robertson et al. (2024) 'Estimating subgroup effects in generalizability and transportability analyses' *Am J Epidemiol*
- Klose et al. (2025) 'Revisiting the Population Attributable Fraction' *Epidemiol*
- Zivich & Lu (2025) 'Constructing g-computation estimators: two case studies in selection bias' *Epidemiol*

Data fusion

- Cole et al. (2023) 'Illustration of 2 fusion designs and estimators' *Am J Epidemiol*
- Shook-Sa et al. (2024) 'Fusing trial data for treatment comparisons: single versus multi-span bridging' *Stat Med*

Section 1: introduction

Break (15min)

Section 2: applied examples

Break (15min)

Section 3: in context

Section 1: introduction

Break (15min)

Section 2: applied examples

Break (15min)

Section 3: in context

Overview: Section 1

Review notation / definitions

Estimating equations by-hand

Estimating equations with a computer

Some statistical properties

Review notation and mathematical operations used

- If unfamiliar with something, don't worry!
- Operations will be
 - Contextualized in following sections
 - Mainly done by the computer
- Resource for you to return to later

What we need:

- Basics
- Matrix algebra
- Derivatives

O_i : observed data for unit i

- $O_i = (X_i, Y_i)$

$\sum_{i=1}^n(i) = 1 + 2 + \dots + n$: cumulative sum

$\prod_{i=1}^n(i) = 1 \times 2 \times \dots \times n$: cumulative product

$\text{expit}(a) = 1/(1 + \exp(-a))$

$E[X]$: expected value function

- For a binary X , $E[X] = \Pr(X = 1)$

Notation – Basics

estimand
(parameter of interest)

 θ 

estimator

 $\hat{\theta}$

Ingredients

150g unsalted butter, plus extra for greasing

150g plain chocolate, broken into pieces

150g plain flour

1/2 tsp baking powder

1/2 tsp bicarbonate of soda

200g light muscovado sugar

Method

1. Heat the oven to 160C/140C fan/gas 3. Grease and base line a 1 litre heatproof glass pudding basin and a 450g loaf tin with baking parchment.

2. Put the butter and chocolate into a saucepan and melt over a low heat, stirring. When the chocolate has all melted remove from the heat.

estimate

 0.5 

2

²Estimand also commonly denoted by θ_0 or θ^*

Notation – Vectors & Matrices

Vector: a list of numbers (or scalars)

$$A = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Matrix: a table of numbers

$$\mathbf{B} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Transpose

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix} \quad \mathbf{A}^T = \begin{bmatrix} a & c & e \\ b & d & f \end{bmatrix}$$

Notation – Matrix Algebra

Dot product (matrix multiplication)

$$\mathbf{A} \mathbf{B} = \mathbf{C}$$

The diagram illustrates the dot product (matrix multiplication) $\mathbf{A} \mathbf{B} = \mathbf{C}$. Matrix $\mathbf{A}$ is shown with elements $a_{11}, a_{12}, \dots, a_{1p}$ highlighted in red. Matrix $\mathbf{B}$ is shown with elements $b_{11}, b_{21}, \dots, b_{p1}$ highlighted in blue. The resulting matrix $\mathbf{C}$ is shown with elements $c_{11}, c_{12}, \dots, c_{1n}$ highlighted in purple. The dot product is shown as $a_{11}b_{11} + a_{12}b_{21} + \dots + a_{1p}b_{p1}$, with the red and blue components highlighted in the terms.

- Number of rows in first matrix must match columns in the second matrix

Notation – Matrix Algebra

Dot product (matrix multiplication)

$$\mathbf{A} \mathbf{B} = \mathbf{C}$$

The diagram illustrates the dot product calculation for matrix multiplication $\mathbf{A} \mathbf{B} = \mathbf{C}$. Matrix $\mathbf{A}$ is shown with elements $a_{11}, a_{12}, \dots, a_{1p}$ in the first row, $a_{21}, a_{22}, \dots, a_{2p}$ in the second row (highlighted in red), and $a_{m1}, a_{m2}, \dots, a_{mp}$ in the m -th row. Matrix $\mathbf{B}$ is shown with elements $b_{11}, b_{12}, \dots, b_{1n}$ in the first column, $b_{21}, b_{22}, \dots, b_{2n}$ in the second column (highlighted in blue), and $b_{p1}, b_{p2}, \dots, b_{pn}$ in the p -th column. Matrix $\mathbf{C}$ is shown with elements $c_{11}, c_{12}, \dots, c_{1n}$ in the first row, $c_{21}, c_{22}, \dots, c_{2n}$ in the second row (with c_{21} highlighted in purple), and $c_{m1}, c_{m2}, \dots, c_{mn}$ in the m -th row. An arrow points from the second row of $\mathbf{A}$ and the first column of $\mathbf{B}$ to the expression $a_{21}b_{21} + a_{22}b_{21} + \dots + a_{2p}b_{21}$, which then points to the element c_{21} in matrix $\mathbf{C}$.

- Number of rows in first matrix must match columns in the second matrix

Notation – Matrix Algebra³

Inverse of 2×2 matrix

$$\mathbf{D} = \begin{bmatrix} w & x \\ y & z \end{bmatrix} \quad \mathbf{D}^{-1} = \frac{1}{wz - xy} \begin{bmatrix} z & -y \\ -x & w \end{bmatrix}$$

- Matrix must have same number of rows and columns

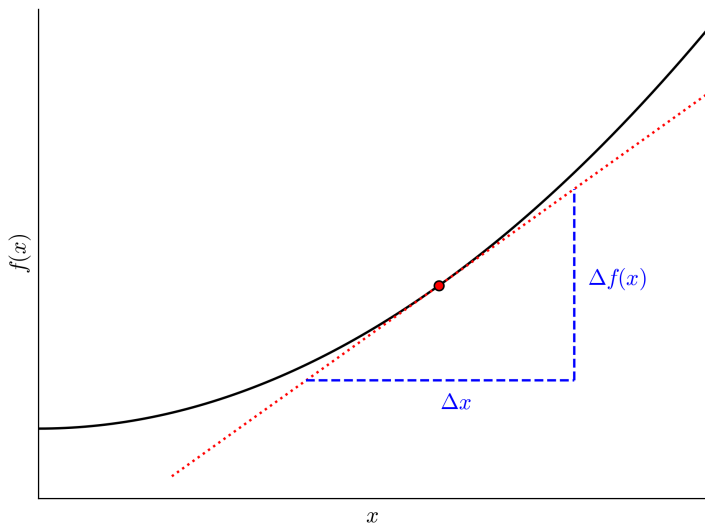
³I've never taken a linear algebra course, so don't worry if this matrix algebra isn't something you're familiar with

$$f'(x) = \frac{d}{dx}f(x)$$

Helpful to think of derivative as slope of tangent line at a point

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Derivatives – Basics



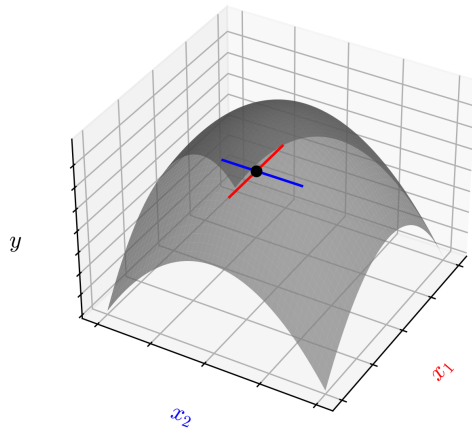
If $\mathbf{x} = (x_1, x_2, \dots, x_m)$ and $f(\mathbf{x}) = y$, then the partial derivative is

$$\frac{\partial}{\partial x_1} f(\mathbf{x})$$

The gradient is

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(\mathbf{x}) \\ \frac{\partial}{\partial x_2} f(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial x_m} f(\mathbf{x}) \end{bmatrix}$$

Derivatives – Generalizations



The Hessian is

$$\Delta H_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2}{\partial x_1 \partial x_1} f(\mathbf{x}) & \dots & \frac{\partial^2}{\partial x_1 \partial x_m} f(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_m \partial x_1} f(\mathbf{x}) & \dots & \frac{\partial^2}{\partial x_m \partial x_m} f(\mathbf{x}) \end{bmatrix}$$

- Jacobian (transpose gradient, ∇^T) of the gradient

Derivatives – Generalization

Function

$$f(x_1, x_2) = y$$

Gradient

$$\nabla f(x_1, x_2) = \begin{bmatrix} \frac{\partial}{\partial x_1} f(x_1, x_2) \\ \frac{\partial}{\partial x_2} f(x_1, x_2) \end{bmatrix}$$

Hessian

$$\Delta H_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2}{\partial x_1 \partial x_1} f(x_1, x_2) & \frac{\partial^2}{\partial x_1 \partial x_2} f(x_1, x_2) \\ \frac{\partial^2}{\partial x_2 \partial x_1} f(x_1, x_2) & \frac{\partial^2}{\partial x_2 \partial x_2} f(x_1, x_2) \end{bmatrix}$$

Notation – Estimating Equations

Estimating *function*

$$\psi(O_i; \theta)$$

Estimating *equation*

$$\sum_{i=1}^n \psi(O_i; \theta)$$

Our estimator, $\hat{\theta}$, is the solution to

k -dimensional estimating function

k -dimensional parameter

$$\sum_{i=1}^n \psi(O_i, \hat{\theta}) = 0$$

Observation i

root: where $f(x) = 0$

Example 0: mean with measurement error

Problem: Learn the Mean

Want to learn the population proportion

- Estimand: $\mu = E[Y] = \Pr(Y = 1)$

But main data only has a mismeasured version of the outcome, Y^*

Have a secondary data source that can estimate sensitivity of Y^*

	Mismeasured Outcome		Validated Outcome
	$Y^* = 0$	$Y^* = 1$	$Y = 1$
Main	120	80	-
Secondary	15	85	100

Assuming specificity is 100%, estimator for the corrected mean is

The diagram shows the formula for the corrected mean estimator: $\hat{\mu} = \frac{\hat{\mu}^*}{\hat{\alpha}}$. The components are color-coded and labeled as follows:

- $\hat{\mu}$ (purple box): Labeled "Corrected mean" with a purple arrow pointing to it.
- $\hat{\mu}^*$ (blue box): Labeled "Naive mean from main" with a blue arrow pointing to it.
- $\hat{\alpha}$ (red box): Labeled "Sensitivity from secondary" with a red arrow pointing to it.

Applying this to the prior data,

$$\hat{\mu}^* = \frac{80}{200} \quad \hat{\alpha} = \frac{85}{100} \quad \hat{\mu} = \frac{0.4}{0.85} = 0.471$$

Estimator – Why It Works

Following the law of total probability,⁴

$$\mu^* = \alpha\mu + (1 - \beta)(1 - \mu)$$

where $\alpha = \Pr(Y^* = 1 \mid Y = 1)$, $\beta = \Pr(Y^* = 1 \mid Y = 0)$,
 $\mu = \Pr(Y = 1)$. Assuming $\beta = 1$,

$$\mu^* = \alpha\mu$$

which can be rewritten as

$$\mu = \frac{\mu^*}{\alpha}$$

⁴Edwards et al. *Am J Epidemiol* 2026; kwag057

An Algorithm for Estimating Equations

1. Determine estimating function
2. Find the roots of the estimating equations
3. Estimate variance via the sandwich

1. Determine Estimating Function

There are several ways to determine an estimating function, but we will use one of the easier approaches: rewrite an estimator as a function equal to zero

Process: starting with

$$\hat{\mu} = \frac{\sum_{i=1}^n Y_i^* R_i}{\sum_{i=1}^n R_i}$$

where R indicates main data. Then re-express as

$$0 = [\text{something}]$$

1. Determine Estimating Function

$$\hat{\mu} = \frac{\sum_{i=1}^n (Y_i^* R_i)}{\sum_{i=1}^n R_i}$$

def'n

$$\hat{\mu}\alpha = \frac{\sum_{i=1}^n (Y_i^* R_i)}{\sum_{i=1}^n R_i}$$

multiply by α

$$\hat{\mu}\alpha \sum_{i=1}^n R_i = \sum_{i=1}^n (Y_i^* R_i)$$

multiply

$$\sum_{i=1}^n (\hat{\mu}\alpha R_i) = \sum_{i=1}^n (Y_i^* R_i)$$

distributive

$$0 = \left(\sum_{i=1}^n Y_i^* R_i \right) - \left(\sum_{i=1}^n \hat{\mu}\alpha R_i \right)$$

subtraction

$$0 = \sum_{i=1}^n (Y_i^* R_i - \hat{\mu}\alpha R_i)$$

associative

1. Determine Estimating Function

The estimating equation of the corrected mean is

The diagram shows the estimating equation $\sum_{i=1}^n \psi(O_i, \hat{\theta}) = \sum_{i=1}^n R_i(Y_i^* - \hat{\mu}\alpha) = 0$. Annotations include: a blue line from 'Estimating function' to ψ ; a purple line from 'Parameter' to $\hat{\theta}$; and a red line from 'Observation i ' to O_i . The terms ψ , O_i , $\hat{\theta}$, R_i , Y_i^* , and $\hat{\mu}\alpha$ are highlighted in light blue boxes.

$$\sum_{i=1}^n \psi(O_i, \hat{\theta}) = \sum_{i=1}^n R_i(Y_i^* - \hat{\mu}\alpha) = 0$$

But this expression assumes we *know* α

- But we don't
- Need another estimating function

1. Determine Estimating Function

$$\hat{\alpha} = \frac{\sum_{i=1}^n (1 - R_i) Y_i^*}{\sum_{i=1}^n (1 - R_i)}$$

def'n

$$\hat{\alpha} \sum_{i=1}^n (1 - R_i) = \sum_{i=1}^n (1 - R_i) Y_i^*$$

multiply

$$\sum_{i=1}^n (1 - R_i) \hat{\alpha} = \sum_{i=1}^n (1 - R_i) Y_i^*$$

distributive

$$0 = \left(\sum_{i=1}^n (1 - R_i) Y_i^* \right) - \left(\sum_{i=1}^n (1 - R_i) \hat{\alpha} \right)$$

subtraction

$$0 = \sum_{i=1}^n ((1 - R_i) Y_i^* - (1 - R_i) \hat{\alpha})$$

associative

$$0 = \sum_{i=1}^n (1 - R_i) \times (Y_i^* - \hat{\alpha})$$

distributive

1. Determine Estimating Function

When we estimated μ by-hand, we did each step separately

However, estimating equations allow us to do both steps simultaneously

- To do this, we 'stack' the estimating functions together

1. Determine Estimating Function

Stacking into a vector

$$\sum_{i=1}^n \begin{bmatrix} R_i(Y_i - \mu\alpha) \\ (1 - R_i) \times (Y_i^* - \alpha) \end{bmatrix}$$

which is becomes our estimator, where $\theta = (\mu, \alpha)$

2. Root-Finding

Given the previous step, how can we find $\hat{\theta}$?

- Ignore the by-hand solutions for the time

Broadly, we can

1. Create a guess for $\hat{\theta}$, denoted by $\tilde{\theta} = (\tilde{\mu}, \tilde{\alpha})$
2. Compute $\sum_{i=1}^n \psi(O_i; \tilde{\theta}) = e$
3. Find the guess $\tilde{\theta}$ where e is close to zero

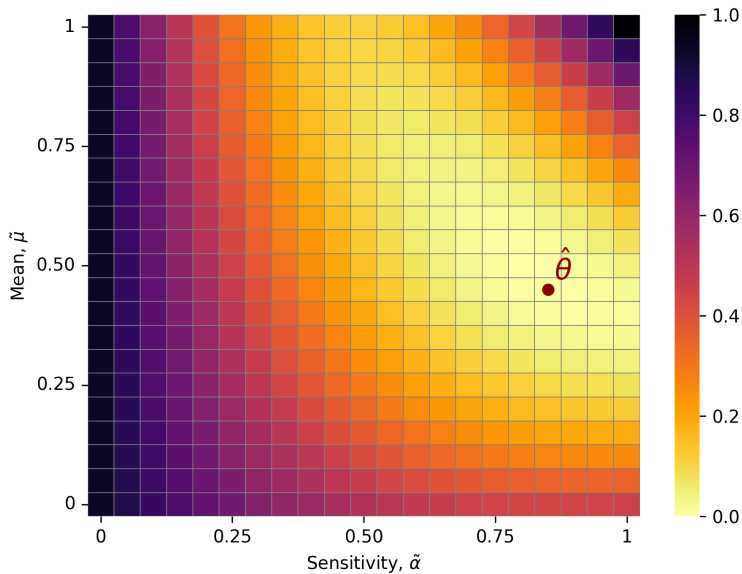
2. Root-Finding

Grid search

- Create a 20-by-20 grid of values for $\tilde{\mu}, \tilde{\alpha}$
- Compute $\sum_{i=1}^n \psi(O_i; \tilde{\theta}) = \begin{bmatrix} e_1 \\ e_2 \end{bmatrix}$
- Compute $e_1^2 + e_2^2$ and find combination closest to zero⁵

⁵Next slide plots the normalized values, so all between zero and one

2. Root-Finding



2. Root-Finding

This process gives us an approximate answer close to our by-hand solution

To reduce this approximation error, we could then

1. Create a new grid around the best values
2. Evaluate that new grid, selecting the best point
3. Repeat process until 'close' to zero

In the next section, will see a better approach than the grid search

3. Variance

For our by-hand calculation, estimating the variance is not easy

- Need to account for variability from both the main data and secondary data
- Uncertainty in α needs to propagate through to μ
- Usually would do something like bootstrapping

But estimating the variance with multiple parameters is easy with estimating equations

- Let's see why and how

3. Sandwich Variance Estimator

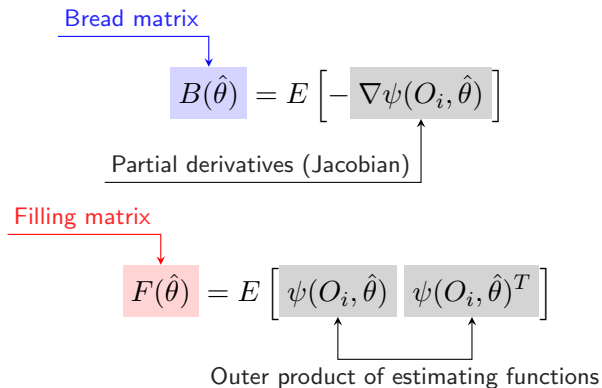
Estimating equations provide the sandwich variance

The diagram illustrates the sandwich variance estimator formula: $V(\hat{\theta}) = B(\hat{\theta})^{-1} F(\hat{\theta}) (B(\hat{\theta})^{-1})^T$. The components are color-coded and labeled as follows:

- Sandwich variance:** A purple label with an arrow pointing to the $V(\hat{\theta})$ term, which is enclosed in a purple box.
- (inverse of) Bread matrix:** A blue label with an arrow pointing to the $B(\hat{\theta})^{-1}$ term, which is enclosed in a blue box.
- Filling (meat) matrix:** A red label with an arrow pointing to the $F(\hat{\theta})$ term, which is enclosed in a red box.

The entire formula is presented as:
$$V(\hat{\theta}) = B(\hat{\theta})^{-1} F(\hat{\theta}) (B(\hat{\theta})^{-1})^T$$

3. Sandwich Variance Estimator



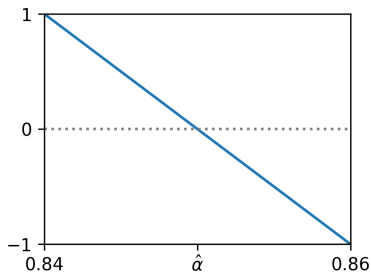
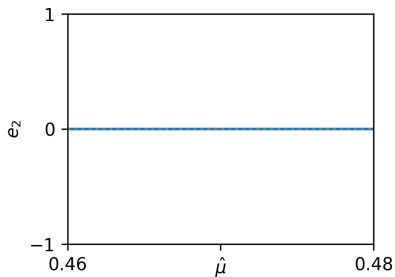
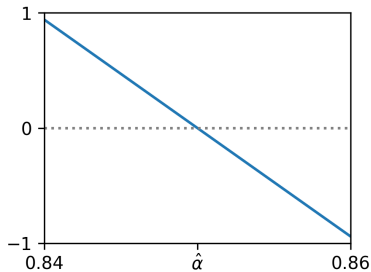
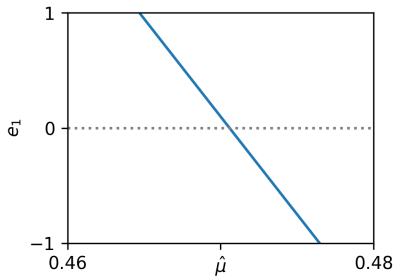
Definition

$$B(\theta) = E \begin{bmatrix} -\frac{\partial}{\partial \mu} R_i (Y_i - \mu \alpha) & -\frac{\partial}{\partial \alpha} R_i (Y_i - \mu \alpha) \\ -\frac{\partial}{\partial \mu} (1 - R_i) (Y_i^* - \alpha) & -\frac{\partial}{\partial \alpha} (1 - R_i) (Y_i^* - \alpha) \end{bmatrix}$$

Won't show the process, but will end up with

$$B(\theta) = E \begin{bmatrix} R_i \alpha & R_i \mu \\ 0 & (1 - R_i) \end{bmatrix} = \begin{bmatrix} E[R_i] \alpha & E[R_i] \mu \\ 0 & E[1 - R_i] \end{bmatrix}$$

Baking the Bread: By-Hands



Definition

$$F(\theta) = E \left(\begin{bmatrix} R_i(Y_i - \mu\alpha) \\ (1 - R_i)(Y_i^* - \alpha) \end{bmatrix} \cdot \begin{bmatrix} R_i(Y_i - \mu\alpha) & (1 - R_i)(Y_i^* - \alpha) \end{bmatrix} \right)$$

Won't show the algebra, but will end up with

$$F(\theta) = E \begin{bmatrix} [R_i(Y_i - \mu\alpha)]^2 & R_i(Y_i - \mu\alpha)(1 - R_i)(Y_i^* - \alpha) \\ (1 - R_i)(Y_i^* - \alpha)R_i(Y_i - \mu\alpha) & [(1 - R_i)(Y_i^* - \alpha)]^2 \end{bmatrix}$$

and since $R_i \times (1 - R_i) = 0$

$$F(\theta) = E \begin{bmatrix} [R_i(Y_i - \mu\alpha)]^2 & 0 \\ 0 & [(1 - R_i)(Y_i^* - \alpha)]^2 \end{bmatrix}$$

Following the reviewed matrix algebra operators

$$V(\hat{\mu}) = \frac{1}{\alpha^2 \Pr(R_i = 1)^2} \times E \left[(R_i(Y_i - \mu\alpha))^2 \right] \\ + \frac{\mu^2}{\alpha^2 \Pr(R_i = 0)^2} \times E \left[((1 - R_i)(Y_i^* - \alpha))^2 \right]$$

where the first term is the variability for μ from the main data, and the second term is the variability contributed from the secondary data when estimating α

Assembling the Sandwich

Wald-type confidence intervals

$$\hat{\mu} \pm z_{\alpha} \sqrt{\frac{\widehat{V}(\hat{\mu})}{n}} = 0.471 \pm 1.96 \sqrt{0.00205/n} = (0.382, 0.559)$$

so the sandwich gives a *valid* estimate of the variance when parameters depend on other parameters⁶

⁶As long as those other parameters are in the stacked estimating functions

The sandwich also gives the variance for sensitivity

$$V(\hat{\alpha}) = \frac{1}{\Pr(R_i = 0)^2} \times E \left[((1 - R_i)(Y_i^* - \alpha))^2 \right]$$

which does not depends on the main data (R_i) or the other estimated proportions

- So the sandwich variance (correctly) ignores the variance of parameters that are not part of an estimating function

Computation of Estimating Equations

Computation of Estimating Equations

Solved estimating equation by-hand

- But quite tedious and lots of math

Consider how estimating equations can be automated

Follow along in `prop_corr.R`, `prop_corr.sas`, or `prop_corr.py`

- Start of code inputs data and sets up estimating equations

Performed a by-hand grid search for $\hat{\theta}$

Variety of multidimensional root-finding algorithms exist⁷

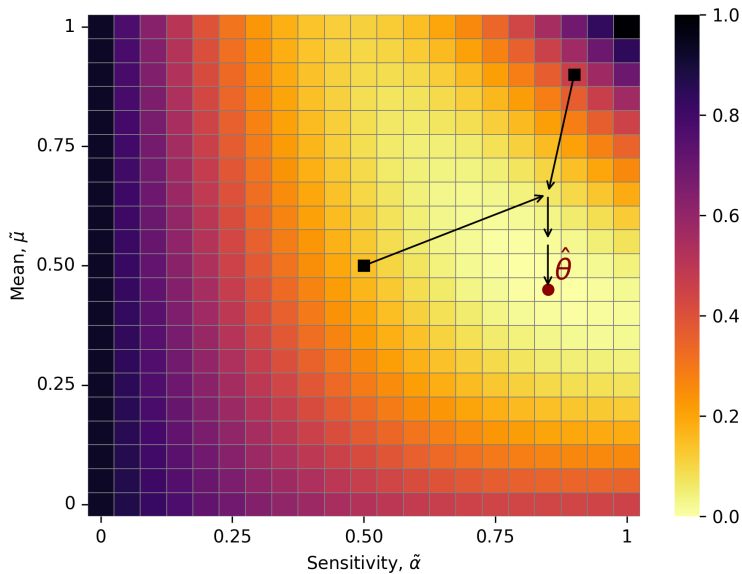
- Secant method (quasi-Newton)
- Levenberg-Marquardt
- Powell hybrid method

⁷I've found Levenberg-Marquardt to be reliable for most problems

Under **Root-finding** see implementation

- SAS – `nlp1m`
- R – `rootSolve::multiroot`
- Python – `scipy.optimize.root`

Root-Finding – Algorithms



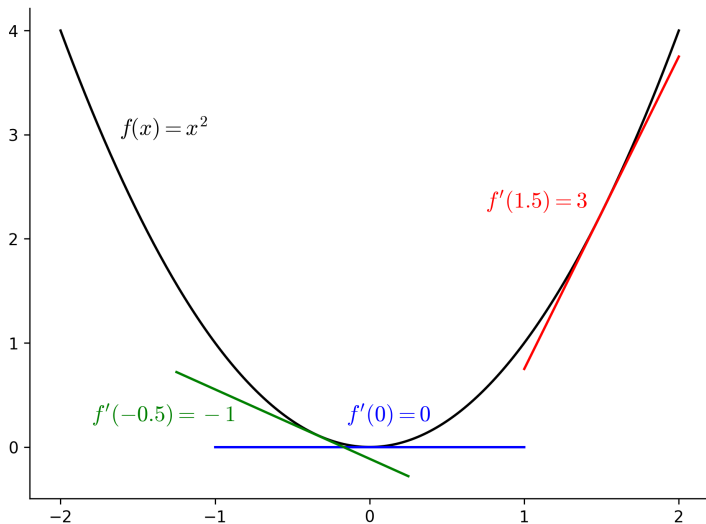
Derivatives – Back to the Definition

The diagram illustrates the definition of a derivative with the following components and annotations:

- Derivative of function**: A black line points from this text to the $f'(x)$ term in the equation.
- Behavior as h becomes small**: A blue line points from this text to the $\lim_{h \rightarrow 0}$ term.
- Change in output (rise)**: A red line points from this text to the numerator $f(x + h) - f(x)$.
- Divided change in input (run)**: A purple line points from this text to the denominator h .

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

Derivatives – Intuition



Central Difference Method⁸

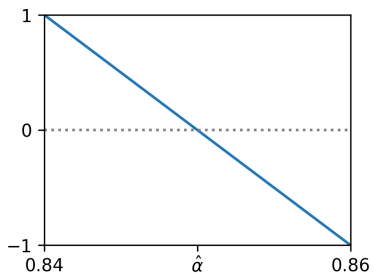
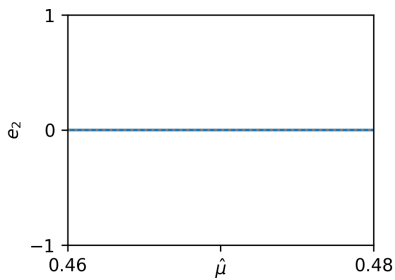
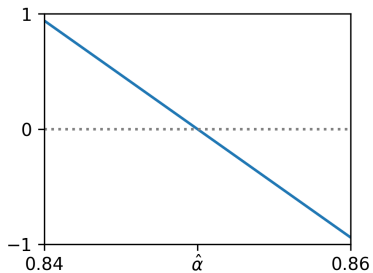
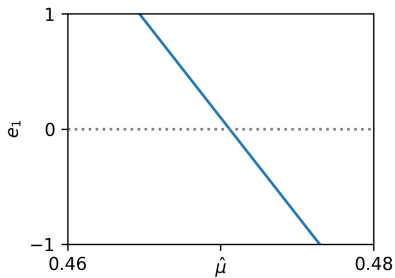
Approximation

$$\tilde{f}'(x) = \frac{f(\overset{\text{Slightly above } x}{x+a}) - f(\overset{\text{Slightly below } x}{x-a})}{2a}$$

Here a is a small value (e.g., 1×10^{-9})

⁸Automatic differentiation, which computes exact derivative, could be used instead. But this is not available in all software and is not straightforward to implement by-hand. Only `delicatessen` offers this option of open-source Estimating Equation libraries

Baking the Bread



Under **Baking the bread** see implementation

- SAS – `nlpfdd`
- R – `numDeriv::jacobian`
- Python – `scipy.optimize.approx_fprime`

Under **Cooking the filling** see implementation

- Transpose
 - SAS – ‘
 - R – `base::t`
 - Python – `numpy.transpose`
- Dot product
 - SAS – *
 - R – `%*%`
 - Python – `numpy.dot`

Under **Assembling the sandwich** see implementation

- Inverse
 - SAS – `inv`
 - R – `base::solve`
 - Python – `numpy.linalg.inv`

Implications of our Algorithm

To evaluate estimating equations, we only need to provide

- Valid estimating functions
- Data

Everything else can be done by the computer

- Simplify complex analyses
- Open-source libraries
 - R: `geex`⁹
 - Python: `delicatessen`¹⁰

⁹Saul & Hudgens (2020) *Journal of Statistical Software*

¹⁰Zivich et al. (2022) *arXiv:2203.11300*

Extensions

Why Estimating Equations?

1. Can straightforwardly build-up complicated estimators by stacking
2. Easily create valid variance estimators
3. Automate all computations via code

As will be further emphasized in later sections, flexibility of the framework is the main advantage

How Estimating Equations are extended

As will be shown further in the next section

1. Stacking estimating functions
2. Automation of delta method

Stacking generalizes to more parameters

$$\sum_{i=1}^n \begin{bmatrix} \psi_{\theta_1}(O_i; \hat{\theta}) \\ \psi_{\theta_2}(O_i; \hat{\theta}) \\ \vdots \\ \psi_{\theta_k}(O_i; \hat{\theta}) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \mathbf{0}$$

- Simple, unlike maximizing a likelihood
 - Likelihood has a single value for individual contribution
 - More difficult to combine likelihood functions

Theorem: smooth function of an asymptotically normal estimator is also asymptotically normal¹¹

Application:

The diagram illustrates the Delta Method formula:
$$Var \left\{ g(\alpha) \right\} \approx g'(\alpha) \Sigma_{\alpha} g'(\alpha)$$
 Annotations include:

- A black arrow labeled "Transformation of α " points from the α in $g(\alpha)$ to the α in $g'(\alpha)$.
- A red arrow labeled "Covariance of α " points from the text to the Σ_{α} term.
- Two blue arrows labeled "Derivative of transformation" point from the text to the $g'(\alpha)$ terms on either side of Σ_{α} .

¹¹Boos & Stefanski *Essential Statistical Inference* pg. 237-240

Many variance formulas you know are Delta method results

- $V(RD)$, $V(\log(RR))$, $V(\log(OR))$
- Formulas follow from Delta method argument
- Don't need to manually solve due to known formulas
 - Not always the case

The estimating function for the transformed parameter, θ_t is

$$\psi_{g(\theta)}(O_i; \theta, \theta_t) = g(\theta) - \theta_t$$

- Estimating function does not depend on data

Therefore, the stacked estimating equations are

$$\sum_{i=1}^n \begin{bmatrix} \psi^*(O_i; \theta) \\ \psi_{g(\theta)}(O_i; \theta, \theta_t) \end{bmatrix} = 0$$

Delta Method with the Sandwich

Following some derivatives and matrix algebra

$$V(\theta, \theta_t) = \begin{bmatrix} V^*(\theta) & g'(\theta)V^*(\theta) \\ V^*(\theta)g'(\theta)^T & g'(\theta)V^*(\theta)g'(\theta) \end{bmatrix}$$

where

$$V(\theta_t) = \begin{matrix} & \text{Sandwich covariance for } \theta \\ g'(\theta) & V^*(\theta) & g'(\theta) \\ \text{Derivative of transformation} \end{matrix}$$

Automate the Delta method!

To close this section, let's discuss the robust variance

- The sandwich variance is also known as the 'robust' variance
- 'Robust' designates that the variance estimator is not sensitive to violations of *certain* assumptions¹²
 - Variance estimator is consistent when parametric model is wrong
 - However this has some difficulties
- Relates back to Maximum Likelihood Estimation
 - The variance can be estimated two ways

¹²See Mansournia et al. (2021) *International Journal of Epidemiology* for further details

Variance estimators

1 Inverse Hessian of the log-likelihood

- Equivalent to $B(\theta)^{-1}$

2 Residuals of the score function

- Equivalent to $F(\theta)^{-1}$

- When the model is correctly specified

- These variance estimators asymptotically equivalent
- $B(\theta) = F(\theta)$

When the model is not correctly specified

- $B(\theta) \neq F(\theta)$
- By combining, sandwich is robust to assumptions
 - Variance estimator is consistent even if model is wrong
- Example: log-Poisson model to estimate the risk ratio
 - Here, estimated variance is too large

Warning¹³

- Does not correct for bias in parameter estimates

¹³See Freedman DA *Am Stat* 2006 for details

Section 1: introduction

Break (15min)

Section 2: applied examples

Break (15min)

Section 3: in context